# Algorithms and Data Structures

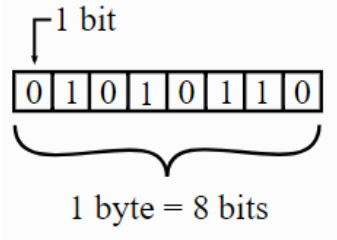
## Encoding
## Character Encoding

**Learning goals**

- ASCII
- 8-Bit
- Unicode

# BITS AND BYTES

- On digital computers, all information is represented in strings of 0's and 1's.
- A single 0/1 digit is called **bit** ("binary" + "digit").
- To represent text, bits are usually arranged in groups of eight, in so-called **bytes**.



# CODES FOR CHARACTER SETS: EXAMPLE ASCII

- The most common code ASCII (American Standard Code for Information Interchange):
  - 95 printable characters.
  - Bit pattern consists of only 7 bits.

Wikipedia:
*ASCII, abbreviated from American Standard Code for Information Interchange, is a character-encoding scheme. ASCII codes represent text in computers, communications equipment, and other devices that use text. Most modern character-encoding schemes are based on ASCII, though they support many additional characters. ASCII was the most common character encoding on the World Wide Web until December 2007, when it was surpassed by UTF-8, which includes ASCII as a subset.*

# CODES FOR CHARACTER SETS: EXAMPLE ASCII

/ 2

*USASCII code chart*

| $b_7$ $b_6$ $b_5$ → | | | | | 0 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1 0 0 | 1 0 1 | 1 1 0 | 1 1 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Bits | $b_4$ | $b_3$ | $b_2$ | $b_1$ | Column → / Row ↓ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| | 0 | 0 | 0 | 0 | 0 | NUL | DLE | SP | 0 | @ | P | ` | p |
| | 0 | 0 | 0 | 1 | 1 | SOH | DC1 | ! | 1 | A | Q | a | q |
| | 0 | 0 | 1 | 0 | 2 | STX | DC2 | " | 2 | B | R | b | r |
| | 0 | 0 | 1 | 1 | 3 | ETX | DC3 | # | 3 | C | S | c | s |
| | 0 | 1 | 0 | 0 | 4 | EOT | DC4 | $ | 4 | D | T | d | t |
| | 0 | 1 | 0 | 1 | 5 | ENQ | NAK | % | 5 | E | U | e | u |
| | 0 | 1 | 1 | 0 | 6 | ACK | SYN | & | 6 | F | V | f | v |
| | 0 | 1 | 1 | 1 | 7 | BEL | ETB | ' | 7 | G | W | g | w |
| | 1 | 0 | 0 | 0 | 8 | BS | CAN | ( | 8 | H | X | h | x |
| | 1 | 0 | 0 | 1 | 9 | HT | EM | ) | 9 | I | Y | i | y |
| | 1 | 0 | 1 | 0 | 10 | LF | SUB | * | : | J | Z | j | z |
| | 1 | 0 | 1 | 1 | 11 | VT | ESC | + | ; | K | [ | k | { |
| | 1 | 1 | 0 | 0 | 12 | FF | FS | , | < | L | \ | l | \| |
| | 1 | 1 | 0 | 1 | 13 | CR | GS | – | = | M | ] | m | } |
| | 1 | 1 | 1 | 0 | 14 | SO | RS | . | > | N | ^ | n | ~ |
| | 1 | 1 | 1 | 1 | 15 | SI | US | / | ? | O | _ | o | DEL |

Source: `https://en.wikipedia.org/wiki/ASCII`

ASCII reserves the first 32 characters for control characters: codes originally intended not to represent printable information, but rather to control devices (printer etc.). For example, the character "LF" represents a line break ("line feed").

# CODES FOR CHARACTER SETS: EXAMPLE ASCII / 3

```
intToBits(65L)
## [1] 01 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00
## [19] 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
utf8ToInt("A")
## [1] 65
```

```
intToUtf8(65)
## [1] "A"
```

```
coderange = c(32:126)
ascii.tab = data.frame(
  char = intToUtf8(coderange, multiple = TRUE),
  dec = coderange,
  hex = as.raw(coderange)
)
```

Note: bit order from left to right (in contrast to the "usual" representation of binary numbers).

# CODES FOR CHARACTER SETS: EXAMPLE ASCII

/ 4

```
head(ascii.tab, 16)
## char dec hex
## 1 32 20
## 2 ! 33 21
## 3 " 34 22
## 4 # 35 23
## 5 $ 36 24
## 6 % 37 25
## 7 & 38 26
## 8 ’ 39 27
## 9 ( 40 28
## 10 ) 41 29
## 11 * 42 2a
## 12 + 43 2b
## 13 , 44 2c
## 14 - 45 2d
## 15 . 46 2e
## 16 / 47 2f
```

# CODES FOR CHARACTER SETS: 8-BIT

There are numerous extensions from ASCII to 8 bit for international character sets:

**ISO 8859 / Latin:** family of standards for different language groups: (1) Western Europe, (2) Eastern Europe, (3) Southern Europe, (4) Northern Europe, (5) Cyrillic, (6) Arabic, (7) Greek, . . .

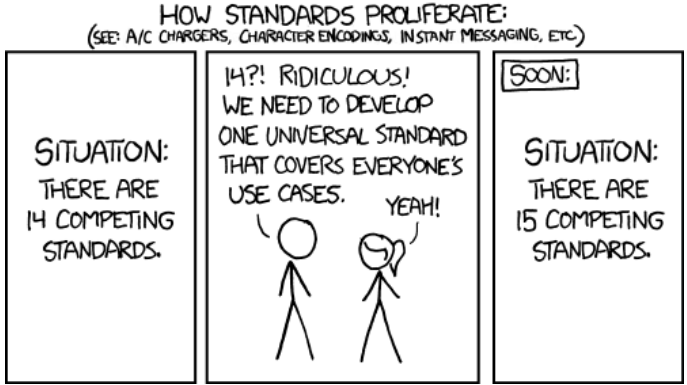
**Windows Code Page 1252:** similar to ISO 8859; however, positions 128-159 are not dedicated to control characters, but also used for printable characters.

Common problems: Sequence of bytes alone are not unique. One needs to know which code is used. Many Asian scripts cannot even be represented in 8 bits.

# CODES FOR CHARACTER SETS: UNICODE

- Goal: a unified code-scheme for any languages, mathematics, music, . . .
- Up to $2^{32}$ characters, probably "only" $2^{21}$ will ever be used. Human languages use the first $2^{16}$ (2 bytes).
- Defined and managed by the unicode consortium, a non-profit organization that all the major hardware & software companies (Adobe, Apple, Google, Microsoft, Oracle, IBM, etc. ) belong to.
- Close cooperation with ISO; development of standards for character codes are delegated to unicode.

# CODES FOR CHARACTER SETS: UNICODE / 2



# CODES FOR CHARACTER SETS: UNICODE / 3

Unfortunately, there are several "Unicode Transformation Formats (UTF)". The two most important ones are:

**UTF-8:** the most common scheme on Unix systems. The Internet Mail Consortium (IMC) recommended that all e-mail programs should be able to display and create mail using UTF-8, and the W3C recommends UTF-8 as the default encoding in XML and HTML.

**UTF-16:** older, is internally used by many "early adopters" like Windows NT (2000, XP, Vista, 7), Java, or Mac OS X. Not compatible with ASCII, since it uses 16 Bit.

# CODES FOR CHARACTER SETS: UNICODE / 4

UTF-8 has a variable width encoding to use 1-4 bytes:

| Bytes | Avail. Bits | Byte 1 | Byte 2 | Byte 3 | Byte 4 |
|---|---|---|---|---|---|
| 1 | 7 | 0xxxxxxx | | | |
| 2 | 11 | 110xxxxx | 10xxxxxx | | |
| 3 | 16 | 1110xxxx | 10xxxxxx | 10xxxxxx | |
| 4 | 21 | 11110xxx | 10xxxxxx | 10xxxxxx | 10xxxxxx |

- First bits in first byte determine the number of total bytes
- UTF-8 with 1 byte is compatible to ASCII